

Sisteme de operare

Laborator 1

Semestrul I 2025

Prezentare generala laborator

- programare de sistem in Linux (clona Unix)
- “Advanced Programming in the Unix Environment”, W. Richard Stevens
- gestiunea fisierelor si I/O
- controlul proceselor (+threads)
- semnale
- IPC (inter-process communication)
 - semafoare System V, monitoare pt pthreads
 - shared memory System V vs POSIX
 - programare client-server (TCP/IP sockets, RPC)
- advanced topics

Laborator 1

- recapitulare concepte ITBI (perspectiva programarii de sistem)
 - procesul de login utilizator
 - interpretorul de comenzi
 - interfata cu nucleul (kernelul) sistemului de operare

Logarea utilizatorului

- *getty* afiseaza prompt-ul de login
- utilizatorul introduce numele de utilizator
- *getty* apeleaza programul */bin/login* cu numele de utilizator ca parametru
- *login*: stabileste o noua sesiune de lucru
 - afiseaza promptul pt parola
 - cauta in */etc/passwd* o intrare corespunzatoare numelui de utilizator
 - verifica parola (de regula stocata criptat in alt fisier, eg. */etc/shadow*)
 - daca parola e corecta
 - executa interpretorul de comenzi (*shell*-ul) asociat intrarii identificate
 - ii asociaza *shell*-ului variabile de mediu (environment) initializate cu valorile din campurile citite din intrarea corespunzatoare din */etc/passwd* (in mod notabil USER, SHELL, HOME)
- *shell*-ul afiseaza un prompt specific (\$) pt utilizator obisnuit) si asteapta comenzile utilizatorului
- *init* monitorizeaza sesiunea de lucru a utilizatorului si cand acesta incheie activitatea (paraseste *shell*-ul), reporneste o instanta a programului *getty* pe terminalul respectiv

Identificarea utilizatorului

- la login, utilizatorul primește un ID propriu (valoare întreaga nenegativă care identifică utilizatorul în sistemul de operare)
 - obținut din intrarea corespunzătoare utilizatorului din */etc/passwd*
 - în general unic
 - asignat de către administratorul sistemului, singurul care are permisiunea de a scrie în */etc/passwd*, și nu poate fi schimbat de către utilizator
 - folosit de kernel pentru a verifica dacă utilizatorul (mai exact procesele sale) are dreptul să execute anumite operații
- utilizatorul cu $UID = 0$ s.n. *root* sau *superuser*
 - intrarea corespunzătoare din */etc/passwd* folosește de regulă numele de *root*
- procesele *root* au privilegii de *superuser* și de regulă circumventează verificările pe care kernelul le face pentru o serie de operații
- unele dintre funcțiile kernelului pot fi executate doar de procese *root*
- *root*-ul are control total asupra sistemului de calcul
 - Obs: din acest motiv, este puternic descurajată inițiativa utilizatorilor sistemului care știu parola de *root* să ruleze programe obișnuite în calitate de *root* ($UID = 0$)

Identificarea utilizatorului (cont.)

- la login utilizatorul nu primește doar un UID ci și un GID (Group ID), setat tot de administrator în intrarea corespunzătoare din */etc/passwd*
 - GID-ul permite partajarea de resurse între membri ai aceluiași grup, chiar dacă au UID-uri diferite
 - De ex, intrările de director pentru fiecare fișier din sistem conțin perechea (UID,GID) a proprietarului fișierului respective
 - comanda shell *ls -l* permite afișarea ID-urilor proprietarului fișierului
 - în schimb, utilizatorii cu GID diferit nu pot accesa aceste resurse partajate ale grupului
- fișierul */etc/group* asignează nume lizibile GID-urilor utilizator;
 - modificabil doar de către administratorul sistemului
 - listează și *supplementary GIDs*, i.e. același utilizator poate avea mai multe GID-uri (poate aparține mai multor grupuri)
- comanda shell cu care se pot afla UID/GID : */usr/bin/id*

Interpretorul de comenzi, shell

- utilizabil deopotriva in mod interactiv cat si batch (folosind *shell script*-uri)
- ofera posibilitatea de a executa atat comenzi interne (executate in cadrul interpretorului) cat si externe (programe incarcate de pe disc)
- functionalitati principale
 - asigurarea unui mediu de lucru utilizatorului (v. comanda *env*)
 - comenzi de manipulare a fisierelor si directoarelor
 - comenzi de control al executiei programelor
 - controlul si monitorizarea activitatilor de I/O
 - administrarea sistemului (rezervata unui utilizator special cunoscut in mod uzual sub numele de *root*, cu UID = 0, v. prima intrare din */etc/passwd*)
 - *samd.*
- ex. interpretoare de comenzi: Bourne Shell (*/bin/sh*), Bourne Again Shell (*/bin/bash*), C Shell (*/bin/csh*), Korn Shell (*/bin/ksh*), etc.

/etc/shells contine interpretoarele de comenzi disponibile in sistem

The Bourne-Again SHell

- */bin/bash*, urmasul primului shell istoric, */bin/sh* (Bourne Shell)
- fisiere de configurare
 - fisiere de start-up inspectate doar la login
 - system-wide: */etc/profile*
 - locale, in home directory: *~/.profile*, *~/.bash_profile*, *~/.bash_login*
 - continutul lor e executat automat la login
 - fisiere de start-up inspectate la crearea fiecarui terminal (rc file, “run commands”)
 - *~/.bashrc*
 - continutul lor poate fi executa voluntar cu comanda *source* (sau “.”)

```
source .bashrc
```

```
..bashrc
```
 - fisiere de logout: *~/.bash_logout*
 - continutul este executat la iesirea din shell (cu *exit* sau *^D*)
 - Obs: ctrl-D in Unix este caracterul EOF, cand utilizatorul tipareste *^D* shell-ul se termina
- istoria comenzilor este inregistrata in *~/.bash_history*

Mediul de lucru (environment)

- o lista de perechi *nume = valoare*
- stocare de shell si disponibile programelor executate din shell
- comanda */usr/bin/env* afiseaza variabilele de mediu
- intr-un program C, variabilele de mediu sunt accesibile in cel de-al treilea parametru al functiei *main*

```
int main(int argc, char* argv[], char *envp[])
```

- cand shell-ul executa programul C, ii furnizeaza mediul de lucru in cel de-al treilea argument al functiei *main*

Descriptori de fisiere

- intregi nenegativi folositi pentru identificarea fisierelor deschise in sistem
- cand kernelul deschide un fisier existent sau creaza unul nou ca urmare a solicitarii unui proces intoarce un descriptor de fisier care va putea fi folosit de proces pentru citirea/scrierea fisierului
- descriptori speciali
 - la pornirea oricarui program, shell-ul deschide pentru acesta trei descriptori de fisiere speciali:
 - 0 standard input
 - 1 standard output
 - 2 standard error
 - uzual, cei trei descriptori de fisier sunt asociati cu terminalul de login (sau terminalul de lucru, intr-un mediu grafic cu multiple X terminale)
 - pentru a afla care este terminalul asociat unui shell (in general, nu doar terminalul de login) se foloseste comanda `/usr/bin/tty`

Apeluri sistem

- toate procesele din sistem (inclusive comenzile shell, interne sau externe) pot solicita serviciile kernelului prin intermediul apelurilor sistem (*system calls*)
 - intrari in codul kernel (nr limitat)
 - apelabile in mod uzual prin intermediul bibliotecii *libc* care este linkeditata automat (cel mai adesea dinamic) la orice program executabil
 - *libc* contine stub-uri cu acelasi nume ca si apelurile de sistem,
 - stub-urile executa o instructiune privilegiata de tip trap (*syscall*) sau intrerupere software (*int 21h* in Linux)
- dpdv programator, apelurile sistem pot fi asimilate unor simple apeluri de biblioteca
- dpdv dezvoltator de cod kernel diferenta e fundamentala
- ex: *printf/cout* apeleaza *write*, *malloc/new* apeleaza *brk/sbrk*, *rand*
 - *write*, *brk* sunt apelurile sistem (de fapt stub-uri in *libc*)
 - *printf/malloc* sunt apeluri de biblioteca

Tratarea erorilor

- diferenta notabila intre apelurile sistem si apelurile obisnuite de biblioteca dpdv al utilizatorului: *tratarea erorilor*
- apel sistem terminat cu eroare => valoare de return negativa si variabila *errno* setata pe o anumita valoare
 - o valoarea de return zero semnifica in general succes
- programatorul are acces la *errno* in C in mod uzual:

```
#include <errno.h>
```

```
extern int errno;
```

- preferabil, folosirea functiilor de biblioteca de tip *strerror* sau *perror*

```
ret_code = ...      // apel sistem
```

```
if (ret_code < 0)
```

```
    perror("apel sistem eronat"), exit(1); N.B.: 0 = succes, >0 = esec
```

Ex. apeluri sistem: read/write

- *ssize_t read(int fd, void *buf, size_t count);* 
 - intoarce nr de octeti cititi (0 daca s-a ajuns la EOF) in *buf*
 - cand *fd* refera un terminal, se citeste cate o linie (eg, pana la `\n`)

ex: citire de la tastatura

```
char buf[256];  
...  
ret_code = read(0, buf, 256);
```

- *ssize_t write(int fd, void *buf, size_t count);*
 - intoace nr de octeti scrisi

ex: scriere pe ecran (terminal)

```
char buf[256];  
...  
ret_code = write(1, buf, 256);
```